AI Engineering

Comprehensive Guide

RAG • MCP Servers • Agents • Fine-Tuning • Evals • Embeddings & More

---

BSD Design & Management • 2025

Table of Contents

1. Foundations of AI Engineering

AI engineering is the discipline of designing, building, deploying, and maintaining production-grade AI systems. Unlike pure machine-learning research, it focuses on the full lifecycle: data pipelines, model integration, serving infrastructure, evaluation, and observability.

1.1 Core Terminology

Term	Definition
LLM	Large Language Model — a deep neural network trained on text to predict and generate language.
Token	The atomic unit an LLM processes. Roughly 3/4 of a word in English.
Context Window	Maximum tokens an LLM can "see" at once (input + output).
Prompt	The input text sent to the model including instructions, data, and examples.
Completion / Response	The model's generated output token sequence.
Temperature	Sampling randomness. 0 = deterministic; 1+ = creative/random.
Top-p / Top-k	Nucleus and top-k sampling — strategies to constrain token selection.
Embedding	A dense numeric vector that encodes semantic meaning of text/data.
Fine-tuning	Continued training of a pre-trained model on a task-specific dataset.
Inference	Running a trained model to produce output (as opposed to training).

1.2 The AI Engineering Stack

A modern AI system is typically composed of several layers working together:

- Data Layer – ingestion, chunking, embedding, vector storage
- Model Layer – LLM APIs, fine-tuned models, local models (Ollama, llama.cpp)
- Orchestration Layer – LangChain, LlamaIndex, custom chains, agents
- Protocol Layer – MCP servers, OpenAPI tool definitions, function calling
- Application Layer – REST APIs, chat interfaces, dashboards
- Observability Layer – tracing (LangSmith, Langfuse), evals, monitoring

2. Retrieval-Augmented Generation (RAG)

RAG is the most widely adopted pattern in production AI engineering. Instead of relying solely on knowledge baked into model weights, RAG retrieves relevant documents at query time and injects them into the prompt. This keeps answers accurate, up-to-date, and grounded in your own data.

2.1 Why RAG?

Challenge	How RAG Solves It
Problem	RAG Solution
LLM knowledge cutoff	Retrieve from live / updated document store
Hallucinations on private data	Ground the model with verified source chunks
Token limit constraints	Return only the top-k most relevant chunks
Cost of fine-tuning	No retraining — update the vector DB instead
Auditability / citations	Point users to exact source documents

2.2 RAG Pipeline Architecture

Indexing Phase (offline)

1. Load – ingest documents (PDFs, HTML, Markdown, databases, APIs).
2. Split – chunk documents into smaller passages (200–1000 tokens each).
3. Embed – convert each chunk into a vector using an embedding model.
4. Store – persist vectors + metadata in a vector database.

Retrieval Phase (online / per query)

5. Embed the user query using the same embedding model.
6. Search the vector DB for the top-k nearest neighbors (cosine similarity).
7. (Optional) Re-rank results with a cross-encoder for precision.
8. Inject retrieved chunks into the prompt context.
9. Generate the answer with the LLM and return with source citations.

2.3 Chunking Strategies

Chunking is one of the most impactful decisions in a RAG system. Poor chunking degrades retrieval quality no matter how good the rest of the pipeline is.

Strategy	Description
Fixed-size chunking	Split every N tokens. Fast and simple. Can cut sentences mid-thought.
Sentence-based chunking	Respect sentence boundaries. Better semantic coherence.
Recursive text splitting	Tries paragraphs → sentences → words. Used in LangChain.
Semantic chunking	Groups sentences by embedding similarity shift. Highest quality.
Document-aware splitting	Use document structure (headers, sections). Best for PDFs and HTML.
Agentic chunking	LLM decides chunk boundaries. Slow but highly accurate.

2.4 Embedding Models

Embedding quality determines retrieval accuracy. Key metrics are MTEB benchmark scores, dimensionality, and inference cost.

Model	Notes
text-embedding-3-large (OpenAI)	3072 dims. Strong on MTEB. Best for English tasks.
text-embedding-3-small (OpenAI)	1536 dims. Faster & cheaper. Still very competitive.
E5-large-v2 (Microsoft)	Open-source, multilingual-capable. Great for on-premise.
BGE-M3 (BAAI)	Multi-lingual, multi-granularity. Strong for non-English.
Cohere embed-v3	Excellent for cross-lingual retrieval. Supports int8 quantization.
nomic-embed-text	8192 token context. Good for long documents.

2.5 Vector Databases

Database	Best For
Pinecone	Fully managed. Simple API. Auto-scaling. Best for quick production.

Weaviate	Open-source / managed. GraphQL + vector hybrid queries.
Qdrant	High-performance Rust-based. Rich filtering. Self-hostable.
Chroma	Lightweight. Perfect for local dev and small-scale apps.
pgvector	Postgres extension. Zero new infra if you already run PG.
Milvus	Cloud-native, horizontally scalable. Ideal for billions of vectors.
Redis + RediSearch	In-memory speed. Good for caching and real-time RAG.

2.6 Advanced RAG Techniques

HyDE – Hypothetical Document Embedding

Ask the LLM to generate a hypothetical ideal document, embed that, and use it for retrieval. Improves recall for abstract queries.

RAG Fusion

Generate multiple query variations, retrieve for each, then Reciprocal Rank Fusion (RRF) merges and re-ranks the combined result sets.

Self-RAG

The model decides when to retrieve, whether retrieved passages are relevant, and whether the final answer is grounded — all via special reflection tokens.

RAPTOR – Recursive Abstractive Processing

Cluster and summarize leaf nodes, then embed summaries to create a hierarchical tree. Enables answering both fine-grained and high-level questions.

Parent-Document Retrieval

Store small child chunks for high-precision retrieval, but return the full parent section to the LLM for better context.

Production RAG Tip

Benchmark your chunking strategy and embedding model against a representative set of 50–100 gold-standard question-answer pairs from your domain before committing to a vector DB schema. Changing chunking after indexing millions of documents is costly.

3. Model Context Protocol (MCP) Servers

The Model Context Protocol (MCP) is an open standard introduced by Anthropic in late 2024 that defines how AI models communicate with external tools, data sources, and services. Think of MCP as USB-C for AI: a universal connector that lets any compliant client talk to any compliant server.

3.1 Why MCP Matters

Before MCP, each AI application had to implement its own custom integration layer for every tool. MCP standardises this so a single server implementation can be used across Claude, Cursor, VS Code Copilot, and any other MCP-compatible client.

3.2 Architecture

Component	Role
MCP Host	The application running the AI model (e.g. Claude Desktop, an IDE).
MCP Client	Protocol client inside the host; manages connections to servers.
MCP Server	Lightweight process that exposes Tools, Resources, and Prompts.
Transport Layer	Stdio (local process) or HTTP + Server-Sent Events (remote).

3.3 The Three Primitives

Tools

Functions the LLM can call to perform actions or fetch data. Each tool has a name, description, and JSON Schema for its input parameters. The model decides when and how to call them.

Resources

Read-only data sources exposed as URI-addressable content (files, database rows, API responses). Resources are fetched by the client on request, not by the LLM directly.

Prompts

Pre-built prompt templates with optional arguments that the host application can surface to users as slash commands or workflow starters.

3.4 Building an MCP Server (Python SDK)

Install the official SDK:

```
pip install mcp
```

Minimal server exposing a tool:

```
from mcp.server import FastMCP
mcp = FastMCP('my-server')

@mcp.tool()
def get_weather(city: str) -> str:
    """Returns current weather for a city."""
    return fetch_weather_api(city)

if __name__ == '__main__':
    mcp.run()
```

For a TypeScript server using the official @modelcontextprotocol/sdk:

```
npm install @modelcontextprotocol/sdk
```

3.5 MCP Server Categories

Category	Examples
File System	Read/write local files, list directories, search content.
Database	Query PostgreSQL, MySQL, SQLite, or MongoDB with schema-aware tools.
Web Search	Brave Search, Tavily, or SerpAPI integrations for real-time search.
Code Execution	Run Python, JavaScript, or shell commands in a sandboxed environment.
Communication	Send emails, Slack messages, or create calendar events.
Memory / KV Store	Persist facts across conversations in a key-value store.
Version Control	Git operations: commit, diff, branch management via GitHub/GitLab APIs.
Browser Automation	Puppeteer or Playwright-driven web scraping and form interaction.

3.6 Security Considerations

- Always validate and sanitise tool inputs — treat LLM output as untrusted user input.
- Scope permissions: filesystem servers should restrict to specific directories.
- Use OAuth 2.0 for remote MCP servers that access third-party APIs.
- Implement rate limiting to prevent runaway agent loops from exhausting APIs.
- Log all tool calls with request/response payloads for audit trails.

MCP Registry

Anthropic maintains an official registry of community MCP servers at github.com/modelcontextprotocol/servers. As of mid-2025 it includes 100+ verified servers covering databases, cloud providers, productivity tools, and developer utilities.

4. Function Calling & Tool Use

Function calling (also called tool use) lets an LLM request execution of a specific function rather than generating prose. The model outputs a structured JSON object — the host application executes the function and returns the result to the model, which then incorporates it into its response.

4.1 The Call Cycle

10. Developer defines tools as JSON Schema objects with name, description, and parameters.
11. User message is sent to the model along with the tool definitions.
12. Model returns a tool_call object (name + arguments) instead of regular text.
13. Application executes the function and gets the result.
14. Result is sent back to the model as a tool result message.
15. Model generates its final natural language response incorporating the result.

4.2 Writing Effective Tool Descriptions

The description field is read by the LLM to decide when to call a tool. Poor descriptions lead to missed calls or incorrect argument values.

> **Best Practice**
> Write tool descriptions as if explaining to a smart intern who has never seen your codebase. Specify: what the tool does, when to use it (and when NOT to), what each parameter means, what units or formats are expected, and what the return value looks like.

4.3 Parallel Tool Calling

Modern APIs (OpenAI, Anthropic, Gemini) support returning multiple tool calls in a single response. This allows an agent to execute independent operations concurrently — for example, fetching weather for three cities simultaneously. Always handle parallel calls in your application layer.

4.4 Tool Design Patterns

Pattern	Description
Atomic tools	One tool per action. Easy to test, debug, and cache. Preferred default.

Composite tools	Bundle related sub-actions. Reduces round-trips for complex workflows.
Read vs. Write separation	Keep data-fetching tools separate from mutation tools for safety.
Idempotent writes	Design write tools to be safely retryable (same input = same result).
Pagination tools	For large datasets, provide cursor-based pagination params.

5. AI Agents & Agentic Systems

An AI agent is a system where an LLM drives a loop of planning, tool execution, and reflection to complete multi-step tasks autonomously. Agents go beyond single question-answer cycles — they maintain state, react to tool outputs, and self-correct.

5.1 Agent Loop (ReAct Pattern)

The most common agent pattern is ReAct (Reasoning + Acting), which structures each step as:

- Thought – the model reasons about what to do next
- Action – the model calls a tool
- Observation – the tool result is returned
- (repeat until the task is complete or a stop condition is met)

5.2 Agent Architectures

Architecture	Description
Single-agent	One LLM drives the full loop. Simple to build. Scales up to moderate complexity.
Multi-agent (sequential)	Agents hand off to each other in a pipeline. Good for distinct phases.
Multi-agent (hierarchical)	Orchestrator agent delegates to specialist sub-agents. Handles complexity.
Multi-agent (parallel)	Agents run simultaneously; coordinator aggregates results. Fastest for divisible tasks.
Mixture of Experts	Router model selects the best specialised model for each request.

5.3 Memory Types in Agents

Type	Description
In-context memory	Everything in the active prompt window. Fast but limited and ephemeral.
External memory (RAG)	Vector DB lookup at query time. Scales to millions of documents.
Episodic memory	Summaries of past interactions stored and retrieved by agent.
Procedural memory	Learned workflows or code stored as executable artefacts.

Working memory	Scratchpad in the prompt for intermediate reasoning (chain-of-thought).

5.4 Agent Frameworks

Framework	Notes
LangChain / LangGraph	Most popular. LangGraph adds stateful, graph-based multi-agent flows.
LlamaIndex Workflows	Event-driven agentic workflows with first-class RAG integration.
AutoGen (Microsoft)	Conversational multi-agent framework. Good for code-writing agents.
CrewAI	Role-based agents with task delegation. High-level, quick to prototype.
Haystack	Pipeline-oriented. Strong for document processing and RAG agents.
Semantic Kernel	Microsoft SDK. Deep .NET / Azure integration.
Pydantic AI	Type-safe agents with dependency injection. Clean Python patterns.

5.5 Guardrails & Safety

Production agents require strict guardrails to prevent harmful or unintended actions:

- Human-in-the-loop checkpoints for irreversible actions (deletes, financial transactions)
- Maximum iteration limits to prevent infinite loops
- Tool allow-lists: agents should only access tools needed for their role
- Output validation with schemas (Pydantic, Zod) before acting on LLM output
- Sandbox execution: run code in isolated containers (Docker, E2B)
- Prompt injection detection to prevent malicious instructions in retrieved content

6. Prompt Engineering

Prompt engineering is the practice of crafting inputs to LLMs to reliably elicit desired outputs. It is one of the highest-leverage skills in AI engineering — a well-structured prompt can close the gap between a mediocre model and a great one.

6.1 Core Techniques

Zero-shot Prompting

Ask the model to perform a task with no examples. Works well for simple, well-defined tasks with strong models.

Few-shot Prompting

Include 2–10 input/output examples in the prompt to demonstrate the expected format. Dramatically improves performance on structured tasks.

Chain-of-Thought (CoT)

Instruct the model to reason step-by-step before answering ('think step by step'). Significantly boosts accuracy on math, logic, and multi-step reasoning.

Self-Consistency

Generate multiple chain-of-thought responses and take the majority answer. Reduces variance and improves reliability.

Tree of Thoughts (ToT)

Explore multiple reasoning branches, evaluate them, and backtrack when needed. Best for complex search and planning problems.

Role Prompting

Assign the model a persona or expert role in the system prompt. Anchors tone, vocabulary, and reasoning approach.

6.2 System Prompt Architecture

For production systems, the system prompt should be structured and versioned. A common pattern:

- Identity – who the model is and what it does
- Capabilities – what it can and cannot do
- Constraints – hard rules (safety, scope, formatting)
- Context – injected RAG content, user profile, session state
- Output format – JSON schema, Markdown structure, or prose guidelines

6.3 Structured Output

Instruct models to return valid JSON by providing a schema in the prompt or using native structured output modes (OpenAI response_format, Anthropic tool-use pattern). Always validate with Pydantic or Zod before consuming.

6.4 Prompt Versioning & Management

Tool	Use Case
LangSmith Hub	Store, version, and share prompts with team. Integrated with LangChain.
Promptfoo	CLI-based prompt testing and evaluation. Open-source.
Agenta	Full prompt management platform with A/B testing.
Git + YAML files	Simple and portable. Keep prompts as code. Version like everything else.

7. Fine-Tuning

Fine-tuning adapts a pre-trained model to a specific domain, task, or style by continuing training on a curated dataset. It should be considered only after prompt engineering and RAG have been exhausted — fine-tuning is expensive and reduces flexibility.

7.1 When to Fine-Tune

Fine-Tune When...	Avoid When...
Good candidate	Not a good candidate
Consistent output format (structured JSON)	One-off or low-volume tasks
Domain-specific tone or vocabulary	Task solvable with few-shot prompting
Latency-sensitive (smaller fine-tuned model > large prompted model)	Rapidly changing knowledge (use RAG instead)
High-volume production use case (cost savings)	Limited quality training data (<500 examples)

7.2 Fine-Tuning Methods

Full Fine-Tuning

All model weights are updated. Highest quality but requires massive GPU memory. Practical only for models under 7B parameters on consumer hardware.

LoRA – Low-Rank Adaptation

Trains small low-rank adapter matrices injected into each transformer layer. 10–100x fewer trainable parameters than full fine-tuning. The de-facto standard for efficient fine-tuning.

QLoRA

LoRA applied to a 4-bit quantized base model. Enables fine-tuning of 70B parameter models on a single 48GB GPU. Slight quality trade-off vs. full LoRA.

RLHF / DPO

Reinforcement Learning from Human Feedback (RLHF) and Direct Preference Optimization (DPO) align model behavior with human preferences. Used in instruction-tuning and alignment, not typical task fine-tuning.

7.3 Dataset Preparation

16. Define the task precisely and write a schema for training examples.
17. Collect 500–10,000 high-quality input/output pairs (more for complex tasks).
18. Format as JSONL with 'messages' array (system, user, assistant turns).
19. Split: 80% train, 10% validation, 10% held-out test set.
20. Deduplicate, filter low-quality examples, and check for data leakage.

7.4 Tools & Platforms

Platform / Tool	Notes
OpenAI Fine-Tuning API	GPT-4o mini / GPT-4o fine-tuning. Managed, no infra needed.
Hugging Face TRL	Open-source SFT, RLHF, DPO trainers. Most flexible option.
Axolotl	Config-file-driven fine-tuning for open models. Easy QLoRA setup.
Unsloth	2x faster, 80% less VRAM for LoRA. Great for single-GPU setups.
Modal / RunPod	Cloud GPU platforms for training jobs. Pay-per-second.
Together AI	Managed fine-tuning for open models with serving included.

8. Evaluation (Evals)

Evals are automated tests that measure LLM system quality. They are the most important engineering practice for maintaining and improving AI systems over time — without evals, you are flying blind.

8.1 Types of Evals

Type	Description
Unit evals	Test individual prompts or components with known inputs/outputs. Fast.
End-to-end evals	Test the full pipeline from user query to final answer. Slow but comprehensive.
Human evals	A/B preference ratings from real users or labelers. Ground truth but expensive.
LLM-as-judge	Use a stronger model to evaluate outputs. Scalable approximation of human evals.
Retrieval evals	Measure recall and precision of the RAG retrieval step independently.
Adversarial evals	Red-team prompts to test safety, robustness, and jailbreak resistance.

8.2 Key Metrics

- Faithfulness – does the answer stick to the retrieved context (no hallucination)?
- Answer Relevancy – how directly does the response address the question?
- Context Recall – does the retrieval step surface all necessary information?
- Context Precision – are retrieved chunks actually relevant (low noise)?
- BLEU / ROUGE – n-gram overlap scores for structured generation tasks.
- Exact Match (EM) – for classification or extraction tasks with definitive answers.
- Task-specific metrics – pass rates for code, accuracy for QA, F1 for NER, etc.

8.3 Eval Frameworks

Framework	Strength
RAGAS	Open-source RAG evaluation. Measures faithfulness, relevancy, recall, precision.
LangSmith	Full LLMOps platform with dataset management, evals, and tracing.

Promptfoo	CLI tool for running evals across models and prompts. Open-source.
DeepEval	pytest-style unit tests for LLM outputs. Integrates with CI/CD.
Braintrust	Eval platform with datasets, scoring, and experiment comparison.
OpenAI Evals	Framework for building and running standardised eval suites.

8.4 Building an Eval Pipeline

21. Curate a golden dataset: 100–500 representative queries with reference answers.
22. Run evals on every code change that touches prompts, retrieval, or model config.
23. Track metrics over time in a dashboard — don't rely on one-off runs.
24. Set regression thresholds — fail CI if faithfulness drops below 0.85.
25. Supplement automated evals with weekly human spot-checks.

9. Observability & LLMOps

LLMOps covers the operational practices for running AI systems in production: logging, tracing, monitoring, cost tracking, and continuous improvement.

9.1 What to Instrument

- Every LLM call: model, prompt, response, latency, tokens (input/output), cost
- Every tool call: name, arguments, result, latency, success/failure
- Every retrieval: query, top-k chunks returned, similarity scores
- User feedback: thumbs up/down, corrections, follow-up queries
- Errors and retries with full context for debugging

9.2 Platforms

Platform	Best For
LangSmith	Best-in-class tracing for LangChain apps. Dataset and eval management.
Langfuse	Open-source LLMOps. Self-hostable. Great for GDPR-sensitive workloads.
Helicone	Lightweight proxy-based logging for any LLM API. Zero code changes.
Arize Phoenix	ML observability platform with LLM tracing and embedding visualization.
Weights & Biases	Experiment tracking extended to LLM fine-tuning and prompt comparison.
Datadog LLM Observability	Enterprise-grade. Integrates with existing Datadog infra.

9.3 Cost Management

LLM API costs can grow quickly. Key strategies:

- Cache identical or near-identical queries with semantic caching (GPTCache, Redis + embeddings)
- Route simple queries to smaller, cheaper models (GPT-4o mini, Haiku)
- Compress prompts: remove redundant context, summarize long histories
- Batch non-time-sensitive requests (Anthropic Batch API = 50% discount)
- Monitor per-feature and per-user costs with dimension tagging

10. Model Serving & Deployment

Getting AI models into production requires careful choices around hosting, scaling, latency, and reliability.

10.1 Deployment Options

Option	When to Use
Managed API (OpenAI, Anthropic, Google)	Zero infra. Pay-per-token. Best for most products.
Inference providers (Together, Fireworks, Groq)	Open models via API. Often 10x cheaper than frontier APIs.
vLLM (self-hosted)	High-throughput GPU serving. PagedAttention for efficient memory use.
Ollama (local)	Run open models locally in development. Great for privacy.
TGI (Hugging Face)	Docker-based inference server. Optimized for transformer models.
llama.cpp	CPU + GPU inference for quantized models. Ultra-low resource usage.

10.2 Streaming Responses

Always stream LLM responses for user-facing applications. Streaming starts returning tokens immediately, dramatically reducing perceived latency. Implement with server-sent events (SSE) or WebSockets on the frontend.

10.3 Reliability Patterns

- Retry with exponential backoff for transient API errors
- Circuit breaker: fall back to a cached or degraded response if primary model is down
- Model fallback: route to an alternative model when primary is unavailable or slow
- Rate limit handling: implement queuing to smooth out traffic spikes
- Timeout budgets: set hard timeouts per step to prevent request pile-ups

11. AI System Security

AI systems introduce unique attack surfaces beyond traditional web security. Understanding these threats is essential for production deployments.

11.1 Threat Categories

Threat	Description
Prompt Injection	Malicious instructions injected via user input or retrieved content to override system behaviour.
Indirect Prompt Injection	Attack payload embedded in a document, website, or tool result that the agent processes.
Data Exfiltration	Tricking the model into leaking sensitive context window content.
Jailbreaking	Social engineering to bypass safety guidelines and content filters.
Model Inversion	Extracting training data from model outputs (relevant for fine-tuned models).
Supply Chain	Compromised model weights, poisoned training data, or malicious MCP servers.

11.2 Mitigations

- Separate system prompts from user content with structural markers
- Never pass raw user input directly into tool arguments — validate against a schema first
- Treat all retrieved content (RAG chunks, web pages, email) as untrusted data
- Use least-privilege tool permissions — agents should only access what they need
- Implement output filtering for PII, secrets, and sensitive patterns
- Red-team your system with adversarial prompt suites before going to production

12. Quick Reference: Model Landscape

12.1 Frontier Models (API)

Model	Strengths
Claude Opus 4 (Anthropic)	Strongest reasoning. Best for complex agents and long-context tasks.
Claude Sonnet 4 (Anthropic)	Best cost-performance balance. Recommended default for most products.
Claude Haiku 3.5 (Anthropic)	Ultra-fast, low cost. Routing, classification, and simple tasks.
GPT-4o (OpenAI)	Strong multimodal. Excellent structured output support.
o3 (OpenAI)	Extended thinking for hard math and science reasoning.
Gemini 2.0 Pro (Google)	1M token context. Strong for document and video understanding.
Gemini 2.0 Flash (Google)	Fastest Gemini. Low cost. Good multimodal throughput.

12.2 Open Models (Self-Hosted)

Model	Notes
Llama 3.3 70B (Meta)	Best open model for most tasks. Apache 2.0 license for commercial use.
Mistral Large 2	Strong European alternative. Efficient for coding and instructions.
Qwen 2.5 72B (Alibaba)	Top multilingual open model. Excellent code and math.
DeepSeek V3 / R1	State-of-the-art open model. R1 has strong reasoning / CoT.
Phi-4 (Microsoft)	Small but highly capable 14B model. Excellent for edge deployment.
Gemma 2 27B (Google)	Efficient, commercially usable. Good instruction following.

13. Production Architecture Patterns

13.1 The AI Gateway Pattern

Route all LLM calls through a single internal gateway service. This gives you a central place for auth, rate limiting, cost tracking, model routing, caching, and observability. LiteLLM is the most popular open-source implementation.

13.2 Semantic Caching

Cache LLM responses keyed by the semantic similarity of the query rather than exact text match. If a new query is within a cosine similarity threshold of a cached query, return the cached response. Reduces costs by 30–60% for high-traffic applications with overlapping queries.

13.3 Query Router

A fast classifier (small LLM or embedding similarity) routes each incoming query to the optimal handler: direct LLM response, RAG pipeline, tool-calling agent, or cached response. Reduces latency and cost by avoiding heavy pipelines for simple queries.

13.4 Guardrail Middleware

Insert validation layers before and after LLM calls. Input guardrails check for PII, prompt injection, and out-of-scope queries. Output guardrails validate schema compliance, filter sensitive content, and detect hallucinations before returning to the user.

13.5 Asynchronous Agent Execution

For long-running agent tasks, use an async job queue (Celery, BullMQ, or cloud equivalents). The API returns a job ID immediately; the client polls or subscribes via WebSocket for progress updates and the final result. Prevents HTTP timeouts on complex multi-step tasks.